

SOUTHEAST UNIVERSITY

Dhaka, Bangladesh

PROJECT REPORT

Online Shop Management System

This project report CSE282.5: Introduction Programming language II Java

SUBMITTED BY	
Saidur Rahman Joy	2024100000246

SUBMITTED TO
MD Ayon Mia [MDAY] Computer Science & Engineering

Date of Submission: 28th September, 2026

Online Shop Management System

1. Introduction

This project is the Product and Inventory Management component of an Online Shop Management System. It is written in Java and stores product details, checks stock availability, reduces inventory after a sale, validates selected values, and converts product data to and from a delimited string.

2. Objectives

- Create a Product class and product objects.
- Apply the four pillars of OOP where demonstrated by the code.
- Use encapsulation to control access to product data.
- Validate prices and stock quantities.
- Manage stock and handle invalid operations with exceptions.
- Convert product data to a string and recreate an object.

3. Classes and Structure

Product: stores product fields and provides getters, setters, validation, stock operations, display and conversion methods.
OutOfStockException: a custom checked exception used when requested stock exceeds available stock.
ProductDemo: contains main() and tests the Product functionality.

4. Four Pillars of OOP

Encapsulation: Product fields are private. Public getters and setters provide controlled access. Price and stock setters validate values.

Abstraction: Methods such as checkStock(), reduceStock() and displayProduct() provide simple operations while hiding their internal implementation from the caller.

Inheritance: OutOfStockException extends Java's Exception class, reusing the built-in exception structure.

Polymorphism: Product overrides Object's toString() method. Printing a Product object invokes this customized method; this is method overriding, a form of runtime polymorphism.

5. Other OOP and Java Concepts

Class and object: Product is the class; laptop is an object created from it. Constructor: initializes the object's six fields. this keyword: refers to the current object and distinguishes fields from parameters. Methods: separate operations into reusable units. Static method: fromFileString() can be called without an existing Product object. String handling and type conversion: split(), Double.parseDouble() and Integer.parseInt() reconstruct field values.

6. Exception Handling and Validation

setPrice() throws IllegalArgumentException when price is zero or negative. setStockQuantity() rejects negative stock. reduceStock() rejects a non-positive reduction amount and throws OutOfStockException if the requested amount exceeds available stock. The throws clause declares the custom exception, throw raises an exception, and try-catch in ProductDemo handles expected errors and prints messages.

7. Program Demonstration

ProductDemo creates a Laptop product with ID P001, price 750.00 and stock 5. It displays the details and checks availability. Reducing stock by 3 leaves 2 items. A further request to reduce stock by 10 is rejected. The demo then tests invalid price and stock values, converts the product into a delimited string, recreates an object from that string, and displays it.

toFileString() joins product fields using the | delimiter. fromFileString() splits the string, converts numeric values and creates a Product object. These methods demonstrate string conversion only; this version does not actually read from or write to a file. The parser assumes correctly formatted input and does not escape delimiter characters inside fields.

9. Testing Summary

The demonstration covers object creation, product display, stock availability, successful stock reduction, insufficient-stock handling, invalid price, negative stock, string conversion and object reconstruction. Expected stock after the successful sale is 2, and failed operations do not update it.

10. Limitations and Improvements

The constructor assigns price and stock without applying the setter validations. The product ID has a getter but no setter. The string parser has limited input validation. Future improvements could add constructor validation, robust parsing, actual file I/O, product searching, order processing and a user interface.

11. Conclusion

The Online Shop Management System project demonstrates Java classes, objects, encapsulation, abstraction, inheritance, polymorphism, constructors, methods, validation and exception handling through a basic product inventory component. The program provides a foundation for managing product information and preventing selected invalid inventory operations.

Appendix A: Product.java Source Code

```
    }

    }

    }

    }

            );

    }
```

```
public class Product {

private String productId;
private String name;
private double price;
private String category;
private int stockQuantity;
private String description;

public Product(String productId, String name, double price,
String category, int stockQuantity, String description) {

this.productId = productId;
this.name = name;
this.price = price;
this.category = category;
this.stockQuantity = stockQuantity;
this.description = description;
}

public String getProductId() {
return productId;
}

public String getName() {
return name;
}

public double getPrice() {
return price;
}

public String getCategory() {
return category;
}

public int getStockQuantity() {
return stockQuantity;
}

public String getDescription() {
return description;
}

public void setProductId(String productId) {
this.productId = productId;
}

public void setName(String name) {
this.name = name;
}

public void setPrice(double price) {
if (price > 0) {
this.price = price;
}
}

public void setCategory(String category) {
this.category = category;
}

public void setStockQuantity(int stockQuantity) {
if (stockQuantity >= 0) {
this.stockQuantity = stockQuantity;
}
}
```

```
    public void setStockQuantity(int stockQuantity) {
        if (stockQuantity >= 0) {
            this.stockQuantity = stockQuantity;
        }
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public boolean checkStock() {
        return stockQuantity > 0;
    }

public void reduceStock(int amount) throws OutOfStockException {

        if (amount <= 0) {
            return;
        }

        if (amount > stockQuantity) {
            throw new OutOfStockException(
                "Not enough stock available for " + name
            );
        }

        stockQuantity -= amount;
    }

    public void displayProduct() {

        System.out.println("Product ID: " + productId);
        System.out.println("Name: " + name);
        System.out.println("Price: " + price);
        System.out.println("Category: " + category);
        System.out.println("Stock: " + stockQuantity);
        System.out.println("Description: " + description);
    }

    public String toFileString() {

        return productId + "," +
               name + "," +
               price + "," +
               category + "," +
               stockQuantity + "," +
               description;
    }

    public static Product fromFileString(String line) {

        String[] data = line.split(",");
```

```
        if (data.length < 6) {
                return null;
                    }

        return new Product(
                    data[0],
                    data[1],
      Double.parseDouble(data[2]),
                    data[3],
        Integer.parseInt(data[4]),
                    data[5]
                     );
                    }

              @Override
    public String toString() {

  return name + " - $" + price +
      " | Stock: " + stockQuantity;
                    }
                   }


public class OutOfStockException extends Exception {

      public OutOfStockException(String message) {
                          super(message);
                              }
                             }
```